

# 📄 Software Requirements Specification (SRS)

# YumUp — Restaurant Survey Application

---

## ## Document Information

| Field | Details |
|-------|---------|
|-------|---------|

|       |       |
|-------|-------|
| ----- | ----- |
|-------|-------|

|                    |                                           |
|--------------------|-------------------------------------------|
| **Document Title** | Software Requirements Specification (SRS) |
|--------------------|-------------------------------------------|

|                  |                                       |
|------------------|---------------------------------------|
| **Project Name** | YumUp — Restaurant Survey Application |
|------------------|---------------------------------------|

|             |       |
|-------------|-------|
| **Version** | 1.0.0 |
|-------------|-------|

|          |                |
|----------|----------------|
| **Date** | April 28, 2026 |
|----------|----------------|

|            |       |
|------------|-------|
| **Status** | Draft |
|------------|-------|

|                 |                        |
|-----------------|------------------------|
| **Prepared By** | YumUp Development Team |
|-----------------|------------------------|

|                     |                   |
|---------------------|-------------------|
| **Confidentiality** | Internal Use Only |
|---------------------|-------------------|

---

## ## Document Revision History

| Version | Date | Author | Description |
|---------|------|--------|-------------|
|---------|------|--------|-------------|

|       |       |       |       |
|-------|-------|-------|-------|
| ----- | ----- | ----- | ----- |
|-------|-------|-------|-------|

|       |                |          |               |
|-------|----------------|----------|---------------|
| 0.1.0 | April 20, 2026 | Dev Team | Initial draft |
|-------|----------------|----------|---------------|

|       |                |          |                                 |
|-------|----------------|----------|---------------------------------|
| 0.5.0 | April 24, 2026 | Dev Team | Added user stories and UI specs |
|-------|----------------|----------|---------------------------------|

|       |                |          |                    |
|-------|----------------|----------|--------------------|
| 1.0.0 | April 28, 2026 | Dev Team | Final SRS document |
|-------|----------------|----------|--------------------|

---

## ## Table of Contents

...

### 1. Introduction

#### 1.1 Purpose

#### 1.2 Scope

#### 1.3 Definitions & Acronyms

#### 1.4 References

#### 1.5 Overview

### 2. Overall Description

#### 2.1 Product Perspective

#### 2.2 Product Features Summary

#### 2.3 User Classes and Characteristics

#### 2.4 Operating Environment

#### 2.5 Design and Implementation Constraints

#### 2.6 Assumptions and Dependencies

### 3. System Architecture

#### 3.1 High-Level Architecture

#### 3.2 Technology Stack

#### 3.3 Project Structure

#### 3.4 Database Schema

#### 3.5 Authentication Flow

### 3.6 State Management

## 4. Functional Requirements

### 4.1 Authentication Module

### 4.2 Home Module

### 4.3 Meals Module

### 4.4 Rating Module

### 4.5 Suggestions Module

### 4.6 History Module

### 4.7 Profile Module

### 4.8 Notifications Module

### 4.9 Admin Module

## 5. User Stories

### 5.1 Authentication Stories

### 5.2 Meal Browsing Stories

### 5.3 Rating Stories

### 5.4 Suggestion Stories

### 5.5 History Stories

### 5.6 Profile Stories

### 5.7 Admin Stories

## 6. Non-Functional Requirements

### 6.1 Performance

### 6.2 Security

### 6.3 Usability

### 6.4 Reliability

6.5 Scalability

6.6 Compatibility

## 7. User Interface Specifications

7.1 Design System

7.2 Screen Inventory

7.3 Navigation Architecture

7.4 Theme Support

## 8. API & Data Specifications

8.1 Supabase Tables

8.2 Row Level Security Policies

8.3 Realtime Subscriptions

8.4 Storage Buckets

8.5 Edge Functions

## 9. Third-Party Integrations

## 10. Testing Requirements

## 11. Deployment & Release

## 12. Appendices

...

---

## # 1. Introduction

### ## 1.1 Purpose

This Software Requirements Specification (SRS) document provides a complete and detailed description of the requirements for the **\*\*YumUp\*\*** Restaurant Survey Application. It serves as the primary reference for the development team, designers, testers, and stakeholders throughout the software development lifecycle.

This document defines the functional and non-functional requirements, system architecture, user interface specifications, database design, and all technical details necessary to build, test, and deploy the YumUp application.

### ## 1.2 Scope

**\*\*YumUp\*\*** is a cross-platform mobile application built with Flutter and powered by Supabase as the backend-as-a-service (BaaS). The application enables restaurant customers to rate individual meals across multiple dimensions, submit structured suggestions, and track the status of their feedback. Restaurant administrators can monitor meal performance, manage suggestions, and export analytical reports.

#### ### In Scope

- User registration and authentication (Email/Password + Google Sign-In)
- Meal browsing with categories, search, and filtering
- Multi-dimensional meal rating system
- Structured and free-form suggestion submission
- Suggestion status tracking
- User rating history management
- Favorites system

- Push notifications
- Admin dashboard with analytics
- Admin meal management (CRUD)
- Admin suggestion management
- Report generation and export (CSV/PDF)
- Light and Dark theme support
- Multi-language support (English and Arabic)
- Offline rating capability with sync

### ### Out of Scope

- Online ordering or payment processing
- Table reservation system
- Delivery tracking
- Social media features (following users, sharing ratings publicly)
- POS (Point of Sale) integration
- Loyalty or rewards program (Phase 2 consideration)

## ## 1.3 Definitions and Acronyms

| Term | Definition |
|------|------------|
|------|------------|

|       |       |
|-------|-------|
| ----- | ----- |
|-------|-------|

|         |                                     |
|---------|-------------------------------------|
| **SRS** | Software Requirements Specification |
|---------|-------------------------------------|

|          |                      |
|----------|----------------------|
| **BaaS** | Backend as a Service |
|----------|----------------------|

|          |                              |
|----------|------------------------------|
| **CRUD** | Create, Read, Update, Delete |
|----------|------------------------------|

|         |                    |
|---------|--------------------|
| **RLS** | Row Level Security |
|---------|--------------------|

|         |                                         |
|---------|-----------------------------------------|
| **RTL** | Right-to-Left (Arabic layout direction) |
|---------|-----------------------------------------|

| **CTA** | Call to Action (primary button/link) |  
| **UI** | User Interface |  
| **UX** | User Experience |  
| **API** | Application Programming Interface |  
| **JWT** | JSON Web Token |  
| **OTP** | One-Time Password |  
| **CSV** | Comma-Separated Values |  
| **PDF** | Portable Document Format |  
| **Supabase** | Open-source Firebase alternative (PostgreSQL-based BaaS) |  
| **Flutter** | Google's cross-platform UI framework |  
| **Dart** | Programming language used by Flutter |  
| **Riverpod** | State management library for Flutter |  
| **Lottie** | Animation file format for mobile apps |

## ## 1.4 References

| Reference              | Description                                                                                     |
|------------------------|-------------------------------------------------------------------------------------------------|
| Flutter Documentation  | <a href="https://docs.flutter.dev">https://docs.flutter.dev</a>                                 |
| Supabase Documentation | <a href="https://supabase.com/docs">https://supabase.com/docs</a>                               |
| Supabase Flutter SDK   | <a href="https://supabase.com/docs/reference/dart">https://supabase.com/docs/reference/dart</a> |
| Material Design 3      | <a href="https://m3.material.io">https://m3.material.io</a>                                     |
| Riverpod Documentation | <a href="https://riverpod.dev">https://riverpod.dev</a>                                         |

## ## 1.5 Overview

The remainder of this document is organized as follows:

- **Section 2** provides an overall description of the product, its context, and constraints.
- **Section 3** details the system architecture, technology stack, and database design.
- **Section 4** specifies all functional requirements organized by module.
- **Section 5** presents user stories in standard format.
- **Section 6** defines non-functional requirements.
- **Section 7** describes the user interface specifications.
- **Section 8** details API and data specifications.
- **Section 9** covers third-party integrations.
- **Section 10** outlines testing requirements.
- **Section 11** describes deployment and release strategy.
- **Section 12** contains appendices and supplementary information.

---

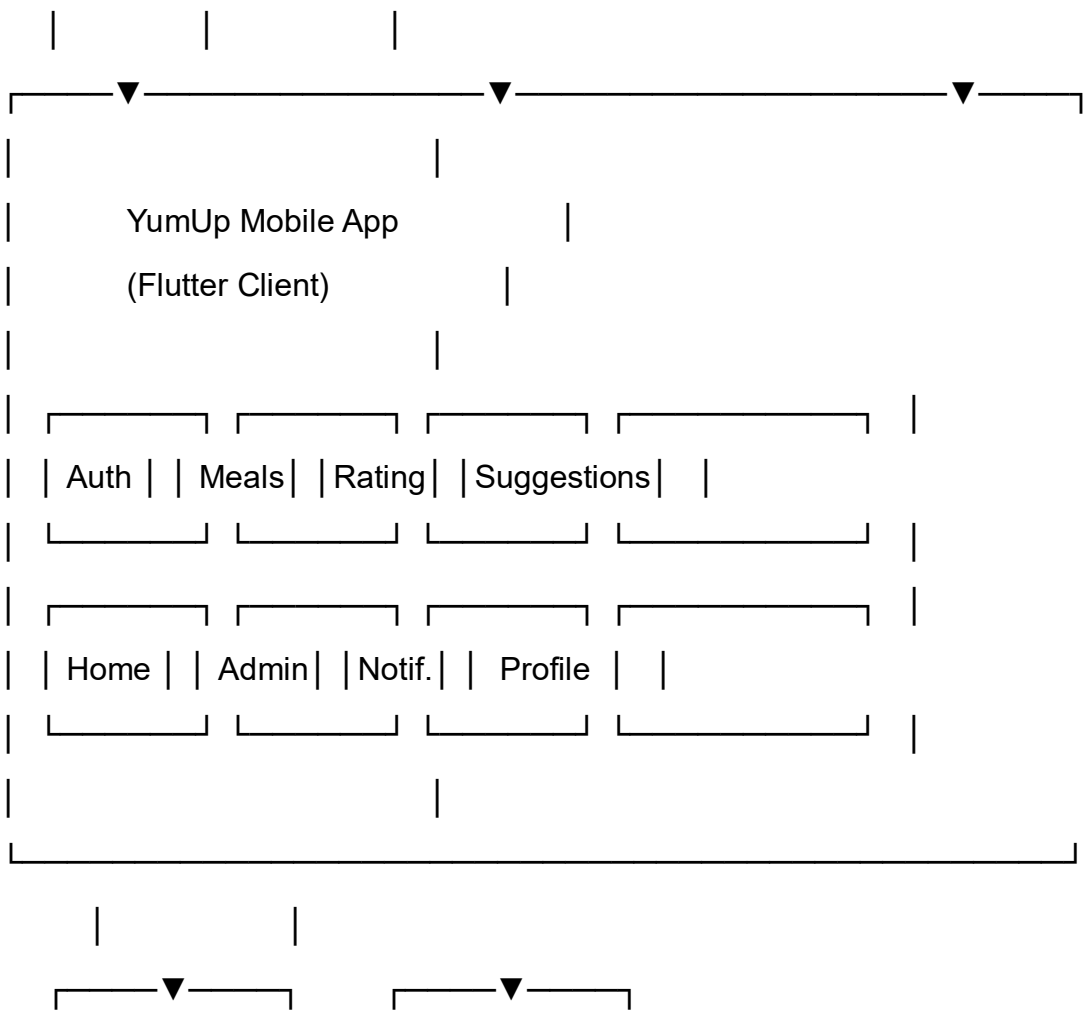
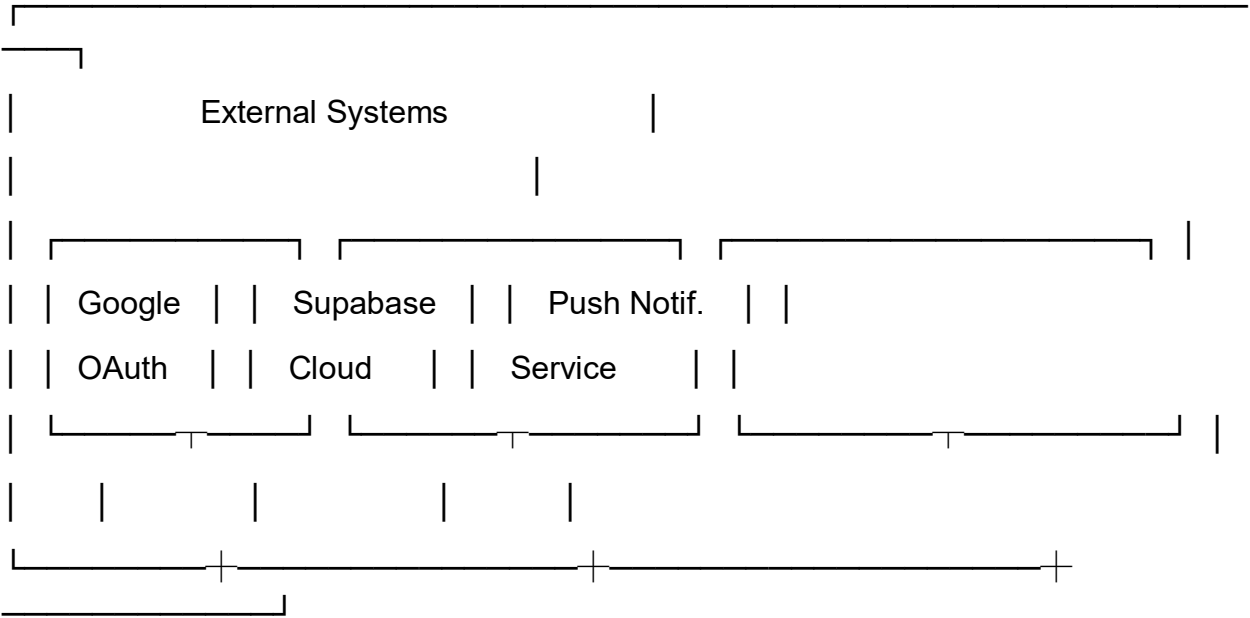
## # 2. Overall Description

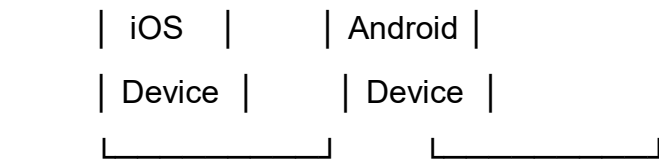
### ## 2.1 Product Perspective

YumUp is a standalone mobile application that operates within a client-server architecture. The mobile client (Flutter) communicates with Supabase cloud services for authentication, database operations, file storage, and real-time data synchronization.

#### ### System Context Diagram

...

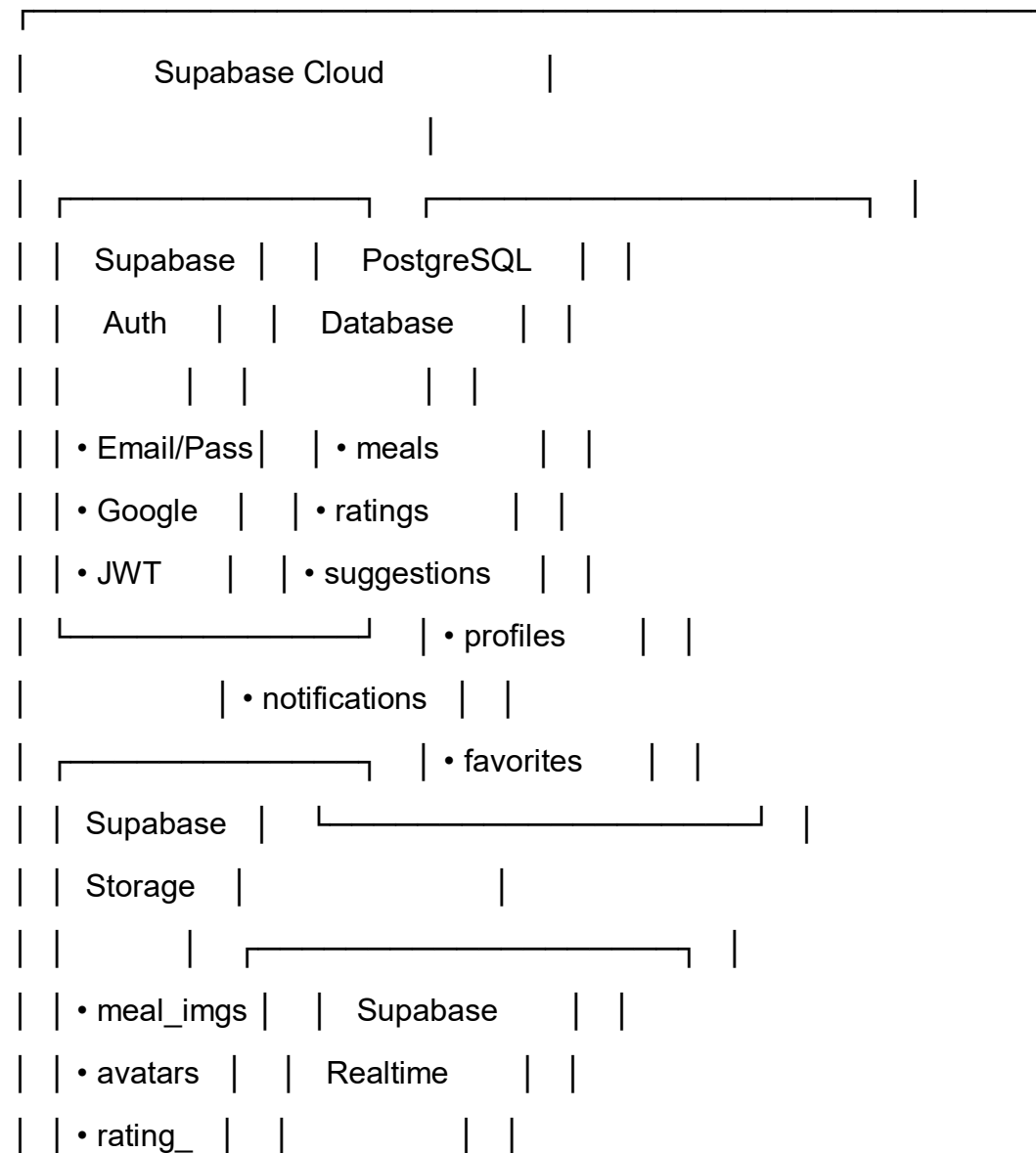


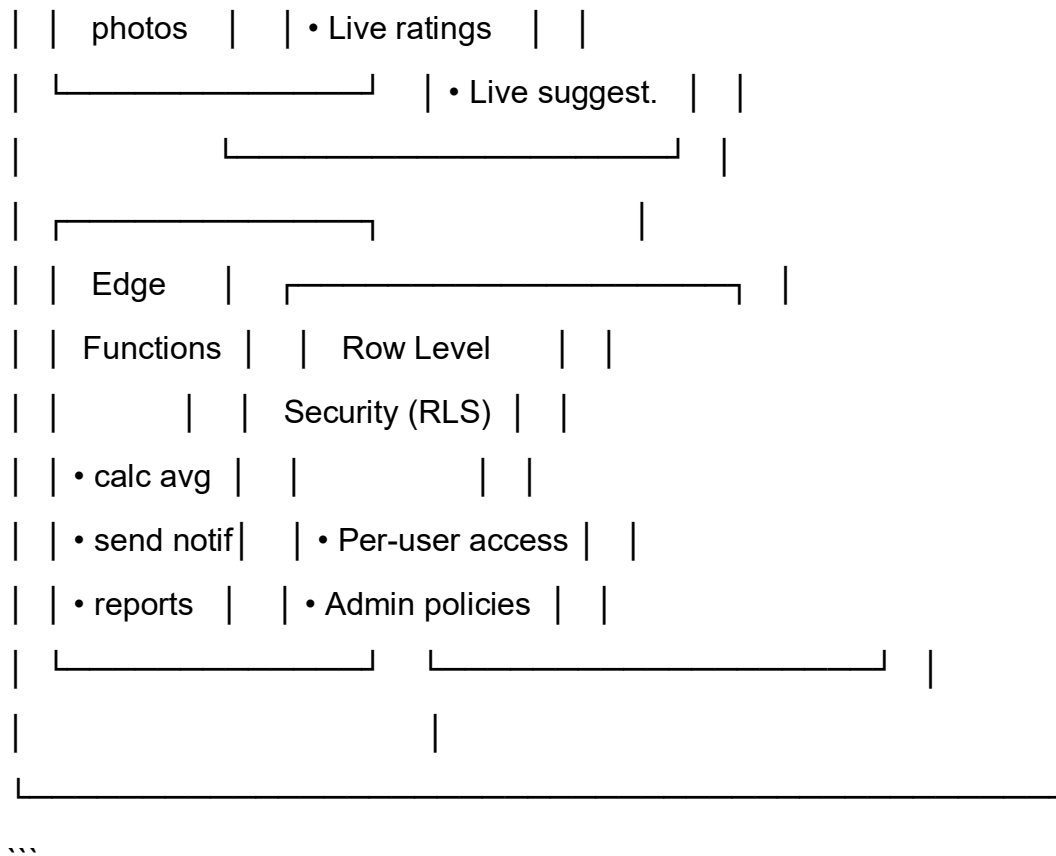


...

### ### Supabase Services Architecture

...





## ## 2.2 Product Features Summary

| Feature ID | Feature Name               | Priority | Description                                              |
|------------|----------------------------|----------|----------------------------------------------------------|
| F-01       | User Authentication        | High     | Email/Password and Google Sign-In registration and login |
| F-02       | Meal Browsing              | High     | Browse, search, and filter meals by category             |
| F-03       | Multi-Dimensional Rating   | High     | Rate meals on taste, presentation, portion, and value    |
| F-04       | Overall Rating Calculation | High     | Auto-calculate overall score from sub-ratings            |
| F-05       | Photo Upload with Rating   | Medium   | Attach a photo when rating a meal                        |
| F-06       | Structured Suggestions     | High     | Submit categorized suggestions to the restaurant         |

|      |                             |        |                                                   |
|------|-----------------------------|--------|---------------------------------------------------|
| F-07 | Suggestion Status Tracking  | Medium | Track suggestion status through a visual timeline |
| F-08 | Rating History              | Medium | View, edit, and delete past ratings               |
| F-09 | Favorites                   | Low    | Save meals to a personal favorites list           |
| F-10 | Notifications               | Medium | Receive push notifications for updates            |
| F-11 | Profile Management          | Medium | Edit profile, change avatar, manage preferences   |
| F-12 | Theme Support               | Medium | Light and Dark mode with user preference          |
| F-13 | Multi-Language              | Medium | English and Arabic with RTL support               |
| F-14 | Offline Support             | Low    | Save ratings offline and sync when connected      |
| F-15 | Admin Dashboard             | High   | Monitor ratings, trends, and top/poor meals       |
| F-16 | Admin Meal Management       | High   | Add, edit, and delete meals                       |
| F-17 | Admin Suggestion Management | High   | Review and update suggestion statuses             |
| F-18 | Admin Reports Export        | Medium | Export reports as CSV or PDF                      |

## ## 2.3 User Classes and Characteristics

### ### User Class 1: Guest User

|                       |                                                      |
|-----------------------|------------------------------------------------------|
| Attribute             | Details                                              |
| -----                 | -----                                                |
| <b>Description</b>    | Unregistered visitor who can browse but not interact |
| <b>Access Level</b>   | Read-only access to public meal listings             |
| <b>Permissions</b>    | View meals, view ratings summary                     |
| <b>Restrictions</b>   | Cannot rate, suggest, or save favorites              |
| <b>Expected Count</b> | 20% of total users                                   |

### ### User Class 2: Registered User

| Attribute | Details |

|-----|-----|

| **Description** | Authenticated customer who actively uses the app |

| **Access Level** | Full user features |

| **Permissions** | Rate meals, submit suggestions, manage favorites, view history |

| **Restrictions** | Cannot access admin features |

| **Expected Count** | 75% of total users |

| **Technical Proficiency** | Basic smartphone usage |

### ### User Class 3: Admin User

| Attribute | Details |

|-----|-----|

| **Description** | Restaurant manager or authorized staff |

| **Access Level** | Full admin access |

| **Permissions** | All user permissions + dashboard, meal CRUD, suggestion management, reports |

| **Restrictions** | None within the application |

| **Expected Count** | 5% of total users (1-5 admins per restaurant) |

| **Technical Proficiency** | Moderate |

### ### User Permissions Matrix

...

|  |  |  |  |  |
|--|--|--|--|--|
|  |  |  |  |  |
|  |  |  |  |  |

| Feature              | Guest | Registered | Admin |
|----------------------|-------|------------|-------|
| View meals           | ✓     | ✓          | ✓     |
| Search meals         | ✓     | ✓          | ✓     |
| View ratings summary | ✓     | ✓          | ✓     |
| Rate a meal          | ✗     | ✓          | ✓     |
| Submit suggestion    | ✗     | ✓          | ✓     |
| Upload photos        | ✗     | ✓          | ✓     |
| Manage favorites     | ✗     | ✓          | ✓     |
| View own history     | ✗     | ✓          | ✓     |
| Edit/delete ratings  | ✗     | Own only   | ✓     |
| Track suggestions    | ✗     | Own only   | ✓     |
| Admin dashboard      | ✗     | ✗          | ✓     |
| Manage meals (CRUD)  | ✗     | ✗          | ✓     |
| Manage suggestions   | ✗     | ✗          | ✓     |
| Export reports       | ✗     | ✗          | ✓     |

...

## ## 2.4 Operating Environment

| Component     | Requirement                         |
|---------------|-------------------------------------|
| **Mobile OS** | Android 6.0 (API 23)+ and iOS 13.0+ |

| **Framework** | Flutter 3.x (latest stable) |  
| **Language** | Dart 3.x |  
| **Backend** | Supabase (PostgreSQL 15+) |  
| **Minimum RAM** | 2 GB |  
| **Storage** | 100 MB app size + user data |  
| **Network** | Internet required (with offline caching support) |  
| **Screen Size** | 4.7 inch to 12.9 inch (phone + tablet) |

## ## 2.5 Design and Implementation Constraints

| Constraint  | Description                                                           |
|-------------|-----------------------------------------------------------------------|
| <b>C-01</b> | Application must be built using Flutter framework                     |
| <b>C-02</b> | Backend services must use Supabase exclusively                        |
| <b>C-03</b> | All data must be stored in Supabase PostgreSQL database               |
| <b>C-04</b> | File storage must use Supabase Storage                                |
| <b>C-05</b> | Authentication must use Supabase Auth                                 |
| <b>C-06</b> | App must support both LTR (English) and RTL (Arabic) layouts          |
| <b>C-07</b> | App must comply with Google Play Store and Apple App Store guidelines |
| <b>C-08</b> | Maximum image upload size: 5 MB per image                             |
| <b>C-09</b> | Rating values must be between 1.0 and 5.0                             |
| <b>C-10</b> | Suggestion text maximum length: 500 characters                        |
| <b>C-11</b> | Review/comment maximum length: 300 characters                         |

## ## 2.6 Assumptions and Dependencies

### ### Assumptions

| ID | Assumption |
|----|------------|
|----|------------|

|--|--|

|      |                                                                         |
|------|-------------------------------------------------------------------------|
| A-01 | Users have a stable internet connection for initial setup and data sync |
|------|-------------------------------------------------------------------------|

|      |                                                       |
|------|-------------------------------------------------------|
| A-02 | Users have basic familiarity with mobile applications |
|------|-------------------------------------------------------|

|      |                                                                                   |
|------|-----------------------------------------------------------------------------------|
| A-03 | The restaurant provides meal data (names, descriptions, images) for initial setup |
|------|-----------------------------------------------------------------------------------|

|      |                                                           |
|------|-----------------------------------------------------------|
| A-04 | Admin users are trained on the admin interface before use |
|------|-----------------------------------------------------------|

|      |                                                                                       |
|------|---------------------------------------------------------------------------------------|
| A-05 | The app will be used primarily within the restaurant premises or shortly after dining |
|------|---------------------------------------------------------------------------------------|

|      |                                                   |
|------|---------------------------------------------------|
| A-06 | One restaurant per app deployment (single-tenant) |
|------|---------------------------------------------------|

### Dependencies

| ID | Dependency | Risk Level |
|----|------------|------------|
|----|------------|------------|

|--|--|--|

|      |                                     |        |
|------|-------------------------------------|--------|
| D-01 | Supabase cloud service availability | Medium |
|------|-------------------------------------|--------|

|      |                                         |     |
|------|-----------------------------------------|-----|
| D-02 | Google OAuth service for Google Sign-In | Low |
|------|-----------------------------------------|-----|

|      |                                   |     |
|------|-----------------------------------|-----|
| D-03 | Flutter SDK stability and updates | Low |
|------|-----------------------------------|-----|

|      |                                 |        |
|------|---------------------------------|--------|
| D-04 | Third-party package maintenance | Medium |
|------|---------------------------------|--------|

|      |                                       |     |
|------|---------------------------------------|-----|
| D-05 | App store review and approval process | Low |
|------|---------------------------------------|-----|

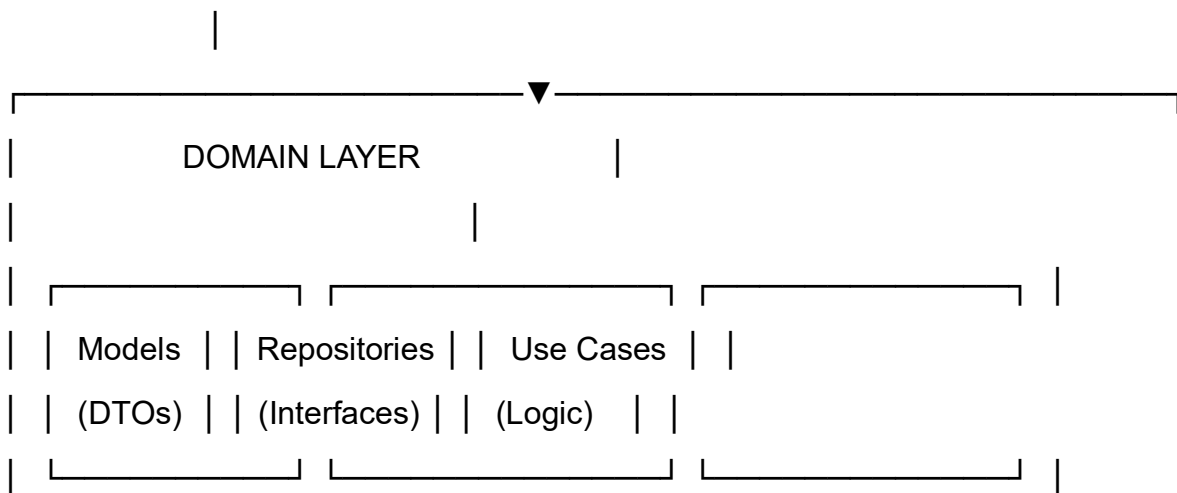
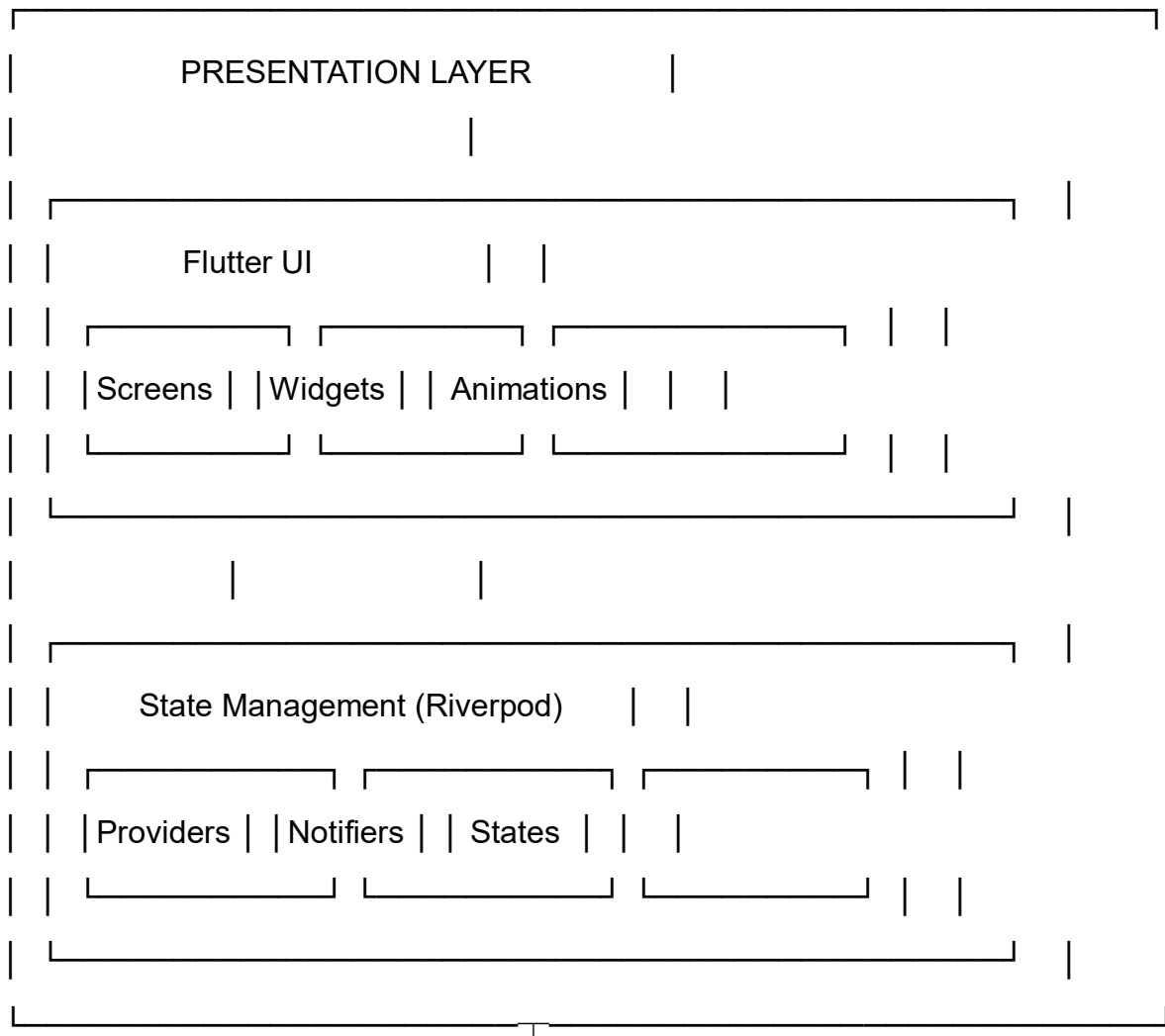
|      |                                        |     |
|------|----------------------------------------|-----|
| D-06 | Push notification service availability | Low |
|------|----------------------------------------|-----|

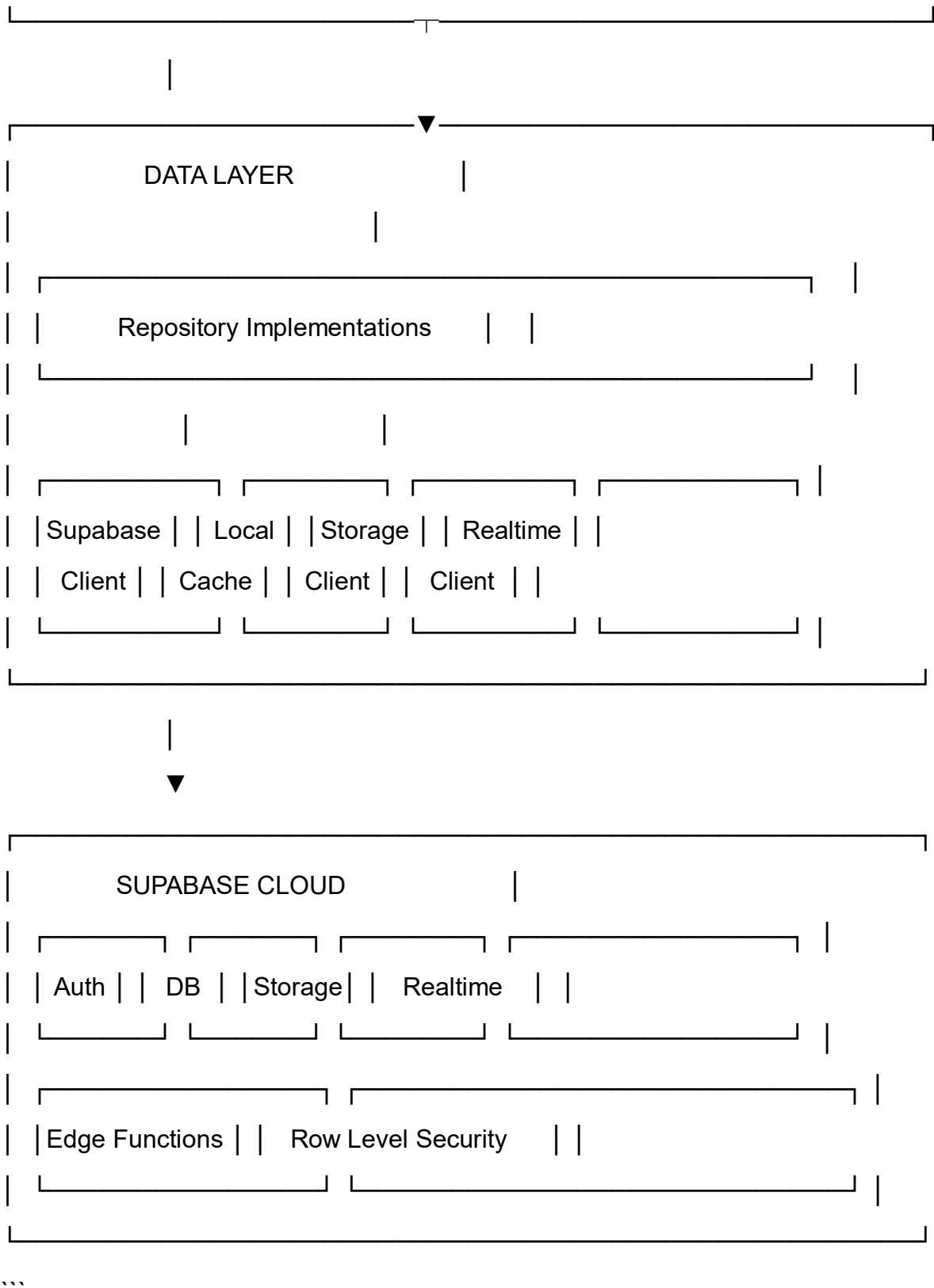
---

## # 3. System Architecture

### ## 3.1 High-Level Architecture

...





## ## 3.2 Technology Stack

### ### Frontend

| Technology      | Version             | Purpose                      |
|-----------------|---------------------|------------------------------|
| Flutter         | 3.x (latest stable) | Cross-platform UI framework  |
| Dart            | 3.x                 | Programming language         |
| Riverpod        | 2.x                 | State management             |
| GoRouter        | 14.x                | Navigation and routing       |
| flutter_animate | 4.x                 | Declarative animations       |
| fl_chart        | 0.68.x              | Charts for admin dashboard   |
| google_fonts    | 6.x                 | Typography (Poppins + Cairo) |

### ### Backend (Supabase)

| Service                 | Purpose                                                 |
|-------------------------|---------------------------------------------------------|
| Supabase Auth           | User authentication (Email, Google OAuth)               |
| Supabase Database       | PostgreSQL database for all app data                    |
| Supabase Storage        | File storage for images (meals, avatars, rating photos) |
| Supabase Realtime       | WebSocket subscriptions for live data updates           |
| Supabase Edge Functions | Server-side logic (calculations, notifications)         |
| Row Level Security      | Data access control per user role                       |

### ### Complete Package List

```
```yaml
```

```
dependencies:
```

```
  # Core
```

```
  flutter:
```

```
    sdk: flutter
```

```
  supabase_flutter: ^2.x
```

```
  # State Management
```

```
  flutter_riverpod: ^2.x
```

```
  riverpod_annotation: ^2.x
```

```
  # Navigation
```

```
  go_router: ^14.x
```

```
  # UI Components
```

```
  google_fonts: ^6.x
```

```
  flutter_svg: ^2.x
```

```
  iconsax: ^0.0.8
```

```
  cached_network_image: ^3.x
```

```
  shimmer: ^3.x
```

```
  flutter_rating_bar: ^4.x
```

```
  carousel_slider: ^4.x
```

```
  flutter_slidable: ^3.x
```

```
  # Animations
```

```
  flutter_animate: ^4.x
```

```
  lottie: ^3.x
```

# Charts (Admin)

fl\_chart: ^0.68.x

# Image

image\_picker: ^1.x

image\_cropper: ^5.x

# Local Storage

shared\_preferences: ^2.x

hive\_flutter: ^1.x

# Utilities

intl: ^0.19.x

connectivity\_plus: ^6.x

url\_launcher: ^6.x

share\_plus: ^9.x

path\_provider: ^2.x

# Export (Admin)

csv: ^6.x

pdf: ^3.x

printing: ^5.x

dev\_dependencies:

flutter\_test:

sdk: flutter

```
riverpod_generator: ^2.x
build_runner: ^2.x
flutter_lints: ^3.x
mockito: ^5.x
...
```

## ## 3.3 Project Structure

```
...

yumup/
├── android/
├── ios/
├── assets/
│   ├── images/
│   │   ├── logo.svg
│   │   ├── onboarding_1.svg
│   │   ├── onboarding_2.svg
│   │   └── onboarding_3.svg
│   ├── lottie/
│   │   ├── splash_loading.json
│   │   ├── confetti_success.json
│   │   ├── empty_state.json
│   │   ├── empty_favorites.json
│   │   └── pull_refresh.json
│   ├── icons/
│   └── fonts/
```

```
|
|
| └─ lib/
|
|   └─ main.dart
|
|   └─ app.dart
|
|   |
|
|   └─ core/
|
|     └─ config/
|
|       └─ app_config.dart
|
|       └─ supabase_config.dart
|
|       └─ env.dart
|
|       |
|
|       └─ theme/
|
|         └─ app_theme.dart
|
|         └─ light_theme.dart
|
|         └─ dark_theme.dart
|
|         └─ app_colors.dart
|
|         └─ app_typography.dart
|
|         └─ app_spacing.dart
|
|         └─ theme_provider.dart
|
|         |
|
|         └─ constants/
|
|           └─ app_constants.dart
|
|           └─ app_assets.dart
|
|           └─ app_strings.dart
|
|           └─ api_constants.dart
```

```
| | |
| | |└── router/
| | |  ├── app_router.dart
| | |  ├── route_names.dart
| | |  └── guards/
| | |      ├── auth_guard.dart
| | |      └── admin_guard.dart
| | |
| | |└── network/
| | |  ├── network_info.dart
| | |  └── connectivity_provider.dart
| | |
| | |└── errors/
| | |  ├── failures.dart
| | |  ├── exceptions.dart
| | |  └── error_handler.dart
| | |
| | |└── utils/
| | |  ├── validators.dart
| | |  ├── date_formatter.dart
| | |  ├── image_helper.dart
| | |  └── export_helper.dart
| | |
| | |
| | |└── shared/
| | |
| | |└── widgets/
| | |  ├── app_button.dart
```

- | | | ├── app\_text\_field.dart
- | | | ├── app\_card.dart
- | | | ├── app\_loading.dart
- | | | ├── app\_error.dart
- | | | ├── app\_empty\_state.dart
- | | | ├── meal\_card.dart
- | | | ├── rating\_bar\_widget.dart
- | | | ├── category\_chip.dart
- | | | ├── status\_badge.dart
- | | | ├── shimmer\_loader.dart
- | | | ├── theme\_toggle.dart
- | | | ├── search\_bar\_widget.dart
- | | | └── bottom\_nav\_bar.dart
- | | |
- | | ├── models/
- | | | ├── user\_model.dart
- | | | ├── meal\_model.dart
- | | | ├── rating\_model.dart
- | | | ├── suggestion\_model.dart
- | | | ├── notification\_model.dart
- | | | └── category\_model.dart
- | | |
- | | └── providers/
- | | ├── supabase\_provider.dart
- | | └── auth\_state\_provider.dart

```
| | └─ connectivity_provider.dart
| |
| └─ features/
|   └─ splash/
|     └─ presentation/
|       └─ splash_screen.dart
|       |
|       └─ onboarding/
|         └─ presentation/
|           └─ onboarding_screen.dart
|           └─ widgets/
|             └─ onboarding_page.dart
|           |
|           └─ auth/
|             └─ data/
|               └─ auth_repository_impl.dart
|               └─ auth_remote_source.dart
|               └─ domain/
|                 └─ auth_repository.dart
|                 └─ auth_use_cases.dart
|                 └─ presentation/
|                   └─ login_screen.dart
|                   └─ register_screen.dart
|                   └─ forgot_password_screen.dart
|                   └─ providers/
|                     └─ auth_provider.dart
```

```
| |   └─ widgets/
| |     └─ login_form.dart
| |     └─ register_form.dart
| |     └─ google_sign_in_button.dart
| |     └─ password_strength.dart
| |
| |   └─ home/
| |     └─ data/
| |       └─ home_repository_impl.dart
| |     └─ domain/
| |       └─ home_repository.dart
| |     └─ presentation/
| |       └─ home_screen.dart
| |       └─ providers/
| |         └─ home_provider.dart
| |       └─ widgets/
| |         └─ greeting_header.dart
| |         └─ promo_banner.dart
| |         └─ categories_section.dart
| |         └─ popular_meals_section.dart
| |         └─ recent_ratings_section.dart
| |
| |   └─ meals/
| |     └─ data/
| |       └─ meals_repository_impl.dart
| |       └─ meals_remote_source.dart
```

```
| | | └─ domain/
| | | | └─ meals_repository.dart
| | | |   └─ meals_use_cases.dart
| | | └─ presentation/
| | |   └─ meals_list_screen.dart
| | |   └─ meal_details_screen.dart
| | |   └─ search_screen.dart
| | |   └─ providers/
| | |     └─ meals_provider.dart
| | |     | └─ meal_details_provider.dart
| | |     └─ widgets/
| | |       └─ meal_list_card.dart
| | |       └─ meal_grid_card.dart
| | |       └─ filter_chips.dart
| | |       └─ sort_dropdown.dart
| | |       └─ rating_breakdown.dart
| | |       └─ review_card.dart
| | |
| | └─ rating/
| | | └─ data/
| | | | └─ rating_repository_impl.dart
| | | |   └─ rating_remote_source.dart
| | | └─ domain/
| | | | └─ rating_repository.dart
| | | |   └─ rating_use_cases.dart
```

```
| | └─ presentation/
| |   └─ rating_screen.dart
| |   └─ rating_success_screen.dart
| |   └─ providers/
| |     └─ rating_provider.dart
| |   └─ widgets/
| |     └─ emoji_rating.dart
| |     └─ star_rating.dart
| |     └─ slider_rating.dart
| |     └─ photo_upload.dart
| |     └─ overall_rating_card.dart
| |     └─ review_text_field.dart
| |
| └─ suggestions/
|   └─ data/
|     └─ suggestion_repository_impl.dart
|     └─ suggestion_remote_source.dart
|   └─ domain/
|     └─ suggestion_repository.dart
|     └─ suggestion_use_cases.dart
|   └─ presentation/
|     └─ suggestion_form_screen.dart
|     └─ suggestion_tracker_screen.dart
|     └─ providers/
|       └─ suggestion_provider.dart
|   └─ widgets/
```



```
| | | | | favorites_provider.dart
| | | | |
| | | | | notifications/
| | | | |
| | | | | data/
| | | | |
| | | | | | | | | notification_repository_impl.dart
| | | | | | | | |
| | | | | | | | | domain/
| | | | | | | | |
| | | | | | | | | | | notification_repository.dart
| | | | | | | | |
| | | | | | | | | presentation/
| | | | | | | | |
| | | | | | | | | | | notifications_screen.dart
| | | | | | | | |
| | | | | | | | | providers/
| | | | | | | | |
| | | | | | | | | | | notification_provider.dart
| | | | | | | | |
| | | | | | | | | profile/
| | | | | | | | |
| | | | | | | | | data/
| | | | | | | | |
| | | | | | | | | | | profile_repository_impl.dart
| | | | | | | | |
| | | | | | | | | domain/
| | | | | | | | |
| | | | | | | | | | | profile_repository.dart
| | | | | | | | |
| | | | | | | | | presentation/
| | | | | | | | |
| | | | | | | | | | | profile_screen.dart
| | | | | | | | |
| | | | | | | | | | | edit_profile_screen.dart
| | | | | | | | |
| | | | | | | | | providers/
| | | | | | | | |
| | | | | | | | | | | profile_provider.dart
| | | | | | | | |
| | | | | | | | | presentation/
| | | | | | | | |
| | | | | | | | | | | profile_header.dart
| | | | | | | | |
| | | | | | | | | | | settings_menu.dart
| | | | | | | | |
```

```
|   └─ admin/
|     └─ data/
|       │   └─ admin_repository_impl.dart
|       │     └─ admin_remote_source.dart
|       └─ domain/
|         │   └─ admin_repository.dart
|         │     └─ admin_use_cases.dart
|         └─ presentation/
|           │   └─ admin_dashboard_screen.dart
|           │   └─ admin_meals_screen.dart
|           │   └─ admin_suggestions_screen.dart
|           │   └─ admin_reports_screen.dart
|           │   └─ providers/
|           │     │   └─ dashboard_provider.dart
|           │     │   └─ admin_meals_provider.dart
|           │     │     └─ admin_suggestions_provider.dart
|           └─ widgets/
|             │   └─ stats_cards.dart
|             │   └─ rating_trend_chart.dart
|             │   └─ top_meals_list.dart
|             │   └─ meal_form_sheet.dart
|             │   └─ suggestion_status_dropdown.dart
|             └─ report_preview_table.dart
|
| └─ test/
```

```

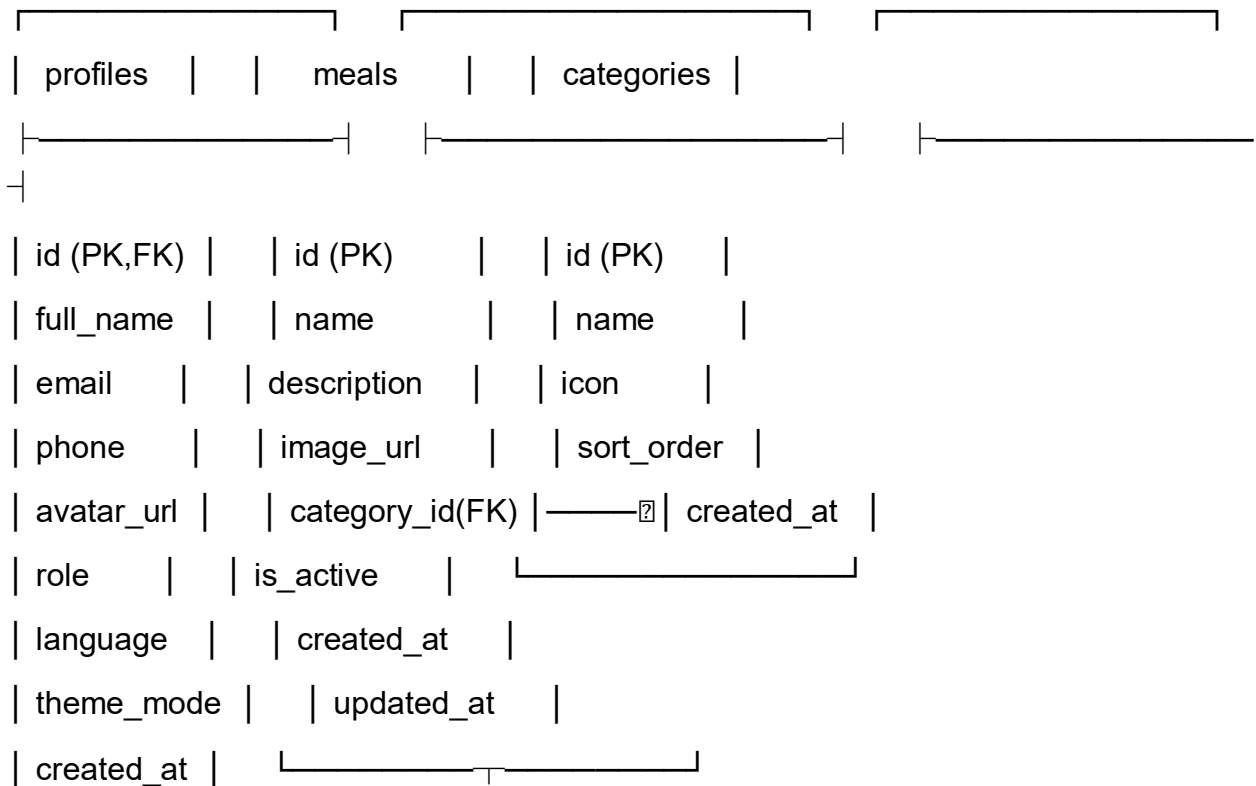
|   ├── unit/
|   ├── widget/
|   └── integration/
|
|   ├── pubspec.yaml
|   ├── analysis_options.yaml
|   └── README.md
...

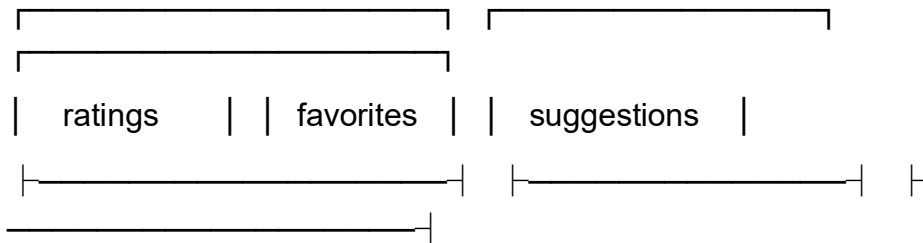
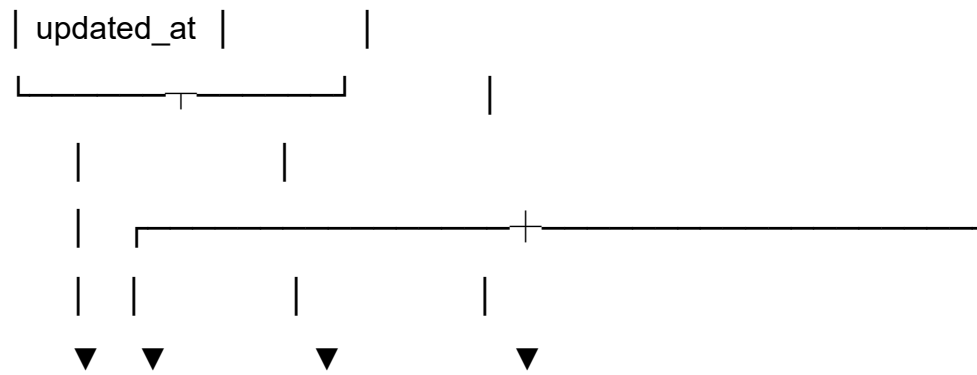
```

## ## 3.4 Database Schema

### ### Entity Relationship Diagram

...





|               |               |              |
|---------------|---------------|--------------|
| id (PK)       | id (PK)       | id (PK)      |
| user_id (FK)  | user_id (FK)  | user_id (FK) |
| meal_id (FK)  | meal_id (FK)  | meal_id (FK) |
| taste_score   | created_at    | type         |
| presentation_ | content       |              |
| score         | priority      |              |
| portion_score | status        |              |
| value_score   | admin_notes   |              |
| overall_score | created_at    |              |
| comment       | updated_at    |              |
| photo_url     |               |              |
| created_at    |               |              |
| updated_at    |               |              |
|               | notifications |              |
|               |               |              |
| id (PK)       |               |              |

|       |              |  |
|-------|--------------|--|
|       | user_id (FK) |  |
|       | title        |  |
|       | body         |  |
|       | type         |  |
|       | reference_id |  |
|       | is_read      |  |
|       | created_at   |  |
| _____ |              |  |

```

### ### Table Definitions

#### #### Table: profiles

```sql

```
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  phone VARCHAR(20),
  avatar_url TEXT,
  role VARCHAR(20) DEFAULT 'user' CHECK (role IN ('user', 'admin')),
  language VARCHAR(5) DEFAULT 'en' CHECK (language IN ('en', 'ar')),
  theme_mode VARCHAR(10) DEFAULT 'light' CHECK (theme_mode IN ('light', 'dark', 'system')),
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW())
```

```
);  
...
```

#### Table: categories

```
```sql
```

```
CREATE TABLE categories (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(50) NOT NULL,  
  name_ar VARCHAR(50) NOT NULL,  
  icon VARCHAR(10) NOT NULL,  
  sort_order INTEGER DEFAULT 0,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMPTZ DEFAULT NOW()  
);
```

-- Seed Data

```
INSERT INTO categories (name, name_ar, icon, sort_order) VALUES  
('Appetizers', '1', '('Main Course', '2', '('Desserts', '3', '('Drinks', '4', '('Salads', '5', '...
```

#### Table: meals

```
```sql
```

```
CREATE TABLE meals (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  name VARCHAR(100) NOT NULL,  
  name_ar VARCHAR(100),  
  description TEXT,  
  description_ar TEXT,  
  image_url TEXT,  
  category_id UUID NOT NULL REFERENCES categories(id) ON DELETE  
RESTRICT,  
  is_active BOOLEAN DEFAULT TRUE,  
  is_featured BOOLEAN DEFAULT FALSE,  
  avg_taste DECIMAL(3,2) DEFAULT 0,  
  avg_presentation DECIMAL(3,2) DEFAULT 0,  
  avg_portion DECIMAL(3,2) DEFAULT 0,  
  avg_value DECIMAL(3,2) DEFAULT 0,  
  avg_overall DECIMAL(3,2) DEFAULT 0,  
  total_ratings INTEGER DEFAULT 0,  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE INDEX idx_meals_category ON meals(category_id);  
CREATE INDEX idx_meals_avg_overall ON meals(avg_overall DESC);  
CREATE INDEX idx_meals_is_active ON meals(is_active);  
...
```

#### Table: ratings

```sql

CREATE TABLE ratings (

id UUID PRIMARY KEY DEFAULT gen\_random\_uuid(),

user\_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,

meal\_id UUID NOT NULL REFERENCES meals(id) ON DELETE CASCADE,

taste\_score DECIMAL(2,1) NOT NULL CHECK (taste\_score BETWEEN 1.0 AND 5.0),

presentation\_score DECIMAL(2,1) NOT NULL CHECK (presentation\_score BETWEEN 1.0 AND 5.0),

portion\_score DECIMAL(2,1) NOT NULL CHECK (portion\_score BETWEEN 1.0 AND 5.0),

value\_score DECIMAL(2,1) NOT NULL CHECK (value\_score BETWEEN 1.0 AND 5.0),

overall\_score DECIMAL(2,1) GENERATED ALWAYS AS (

(taste\_score + presentation\_score + portion\_score + value\_score) / 4.0

) STORED,

comment VARCHAR(300),

photo\_url TEXT,

created\_at TIMESTAMPTZ DEFAULT NOW(),

updated\_at TIMESTAMPTZ DEFAULT NOW(),

UNIQUE(user\_id, meal\_id)

);

CREATE INDEX idx\_ratings\_user ON ratings(user\_id);

```
CREATE INDEX idx_ratings_meal ON ratings(meal_id);
CREATE INDEX idx_ratings_created ON ratings(created_at DESC);
...
```

#### Table: suggestions

```
```sql
```

```
CREATE TABLE suggestions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
  meal_id UUID REFERENCES meals(id) ON DELETE SET NULL,
  type VARCHAR(30) NOT NULL CHECK (
    type IN ('new_dish', 'improve_existing', 'service_issue', 'other')
  ),
  content TEXT NOT NULL CHECK (char_length(content) BETWEEN 10 AND 500),
  priority VARCHAR(10) DEFAULT 'medium' CHECK (
    priority IN ('low', 'medium', 'high')
  ),
  status VARCHAR(20) DEFAULT 'pending' CHECK (
    status IN ('pending', 'under_review', 'in_progress', 'done', 'rejected')
  ),
  admin_notes TEXT,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX idx_suggestions_user ON suggestions(user_id);
```

```
CREATE INDEX idx_suggestions_status ON suggestions(status);
CREATE INDEX idx_suggestions_created ON suggestions(created_at DESC);
...
```

#### Table: favorites

```
```sql
```

```
CREATE TABLE favorites (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    meal_id UUID NOT NULL REFERENCES meals(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ DEFAULT NOW(),

    UNIQUE(user_id, meal_id)
);
```

```
CREATE INDEX idx_favorites_user ON favorites(user_id);
...
```

#### Table: notifications

```
```sql
```

```
CREATE TABLE notifications (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID NOT NULL REFERENCES profiles(id) ON DELETE CASCADE,
    title VARCHAR(100) NOT NULL,
    body TEXT NOT NULL,
```

```
type VARCHAR(30) NOT NULL CHECK (
    type IN ('rating_confirmed', 'suggestion_update',
            'new_meal', 'announcement')
),
reference_id UUID,
is_read BOOLEAN DEFAULT FALSE,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX idx_notifications_user ON notifications(user_id);
CREATE INDEX idx_notifications_read ON notifications(user_id, is_read);
CREATE INDEX idx_notifications_created ON notifications(created_at DESC);
...
```

### ### Database Functions and Triggers

```
```sql
-- Auto-update meal averages after rating insert/update/delete
CREATE OR REPLACE FUNCTION update_meal_averages()
RETURNS TRIGGER AS $$
BEGIN
    UPDATE meals SET
        avg_taste = sub.avg_t,
        avg_presentation = sub.avg_p,
        avg_portion = sub.avg_po,
        avg_value = sub.avg_v,
        avg_overall = sub.avg_o,
```

```

total_ratings = sub.cnt,
updated_at = NOW()
FROM (
    SELECT
        meal_id,
        ROUND(AVG(taste_score), 2) as avg_t,
        ROUND(AVG(presentation_score), 2) as avg_p,
        ROUND(AVG(portion_score), 2) as avg_po,
        ROUND(AVG(value_score), 2) as avg_v,
        ROUND(AVG(overall_score), 2) as avg_o,
        COUNT(*) as cnt
    FROM ratings
    WHERE meal_id = COALESCE(NEW.meal_id, OLD.meal_id)
    GROUP BY meal_id
) sub
WHERE meals.id = sub.meal_id;

RETURN COALESCE(NEW, OLD);
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_update_meal_averages
    AFTER INSERT OR UPDATE OR DELETE ON ratings
    FOR EACH ROW
    EXECUTE FUNCTION update_meal_averages();

-- Auto-update updated_at timestamp

```

```
CREATE OR REPLACE FUNCTION update_timestamp()  
RETURNS TRIGGER AS $$  
BEGIN  
    NEW.updated_at = NOW();  
    RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```

```
CREATE TRIGGER trigger_update_profiles_timestamp  
    BEFORE UPDATE ON profiles FOR EACH ROW  
    EXECUTE FUNCTION update_timestamp();
```

```
CREATE TRIGGER trigger_update_meals_timestamp  
    BEFORE UPDATE ON meals FOR EACH ROW  
    EXECUTE FUNCTION update_timestamp();
```

```
CREATE TRIGGER trigger_update_ratings_timestamp  
    BEFORE UPDATE ON ratings FOR EACH ROW  
    EXECUTE FUNCTION update_timestamp();
```

```
CREATE TRIGGER trigger_update_suggestions_timestamp  
    BEFORE UPDATE ON suggestions FOR EACH ROW  
    EXECUTE FUNCTION update_timestamp();
```

-- Auto-create profile after auth signup

```
CREATE OR REPLACE FUNCTION handle_new_user()  
RETURNS TRIGGER AS $$
```

```

BEGIN
    INSERT INTO profiles (id, full_name, email)
    VALUES (
        NEW.id,
        COALESCE(NEW.raw_user_meta_data->>'full_name', 'User'),
        NEW.email
    );
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

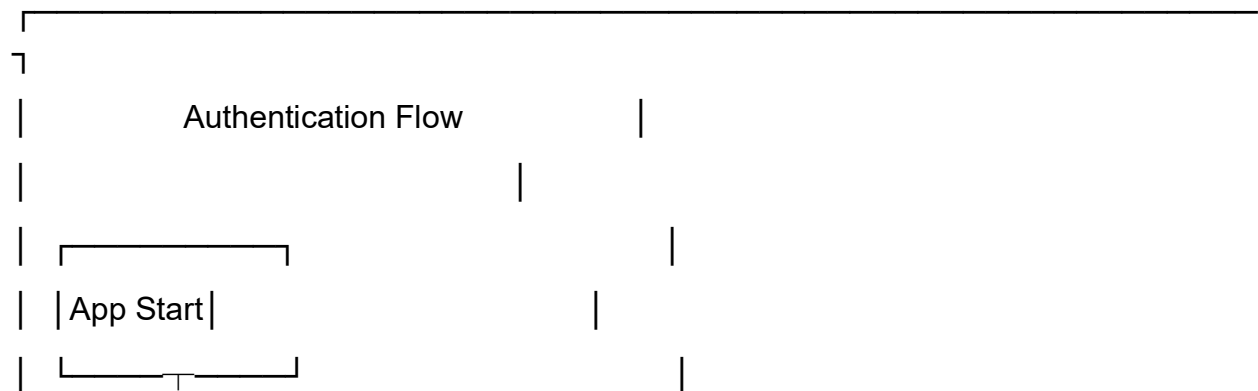
```

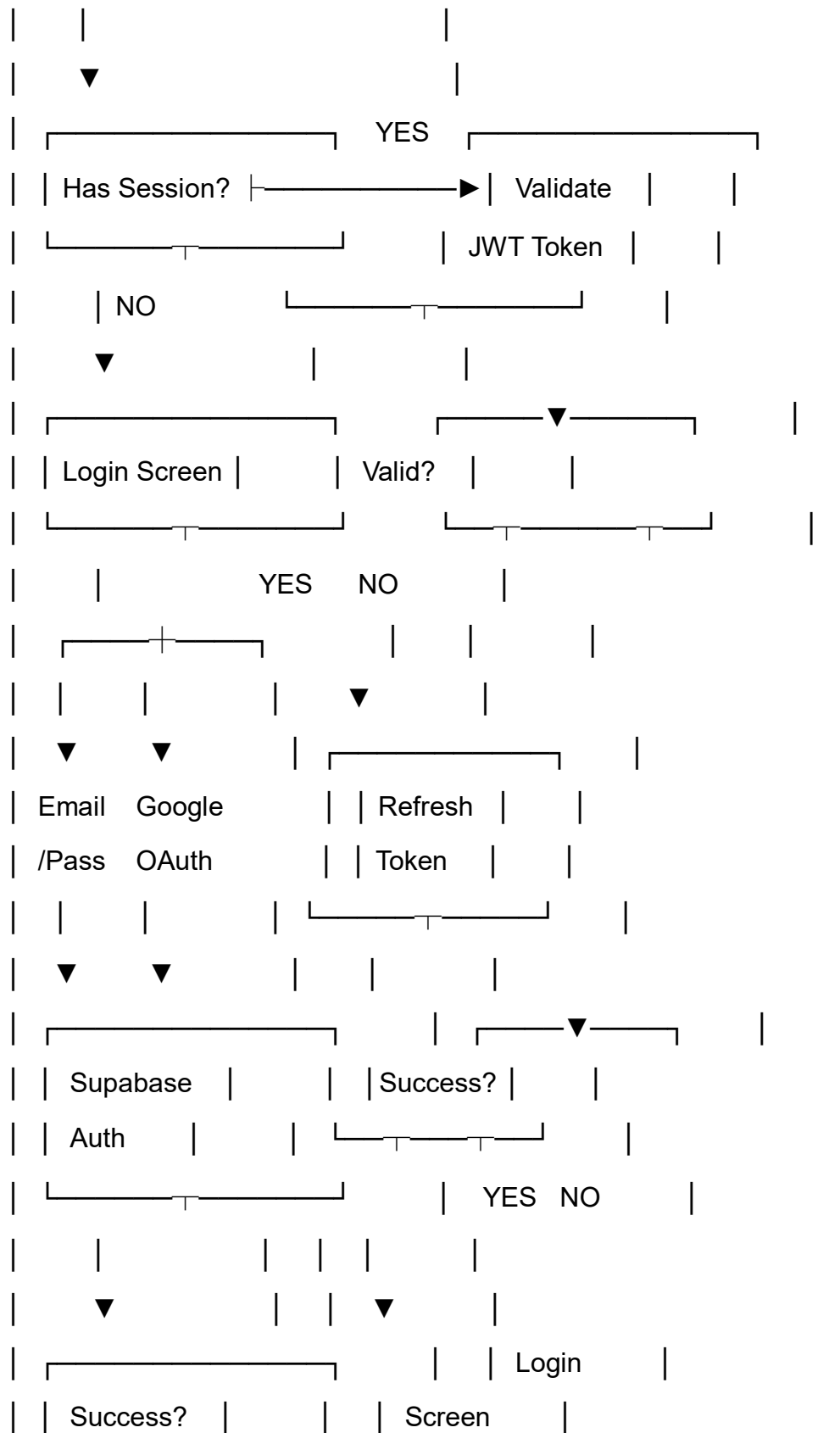
CREATE TRIGGER on_auth_user_created
    AFTER INSERT ON auth.users
    FOR EACH ROW
    EXECUTE FUNCTION handle_new_user();
...

```

## ## 3.5 Authentication Flow

...



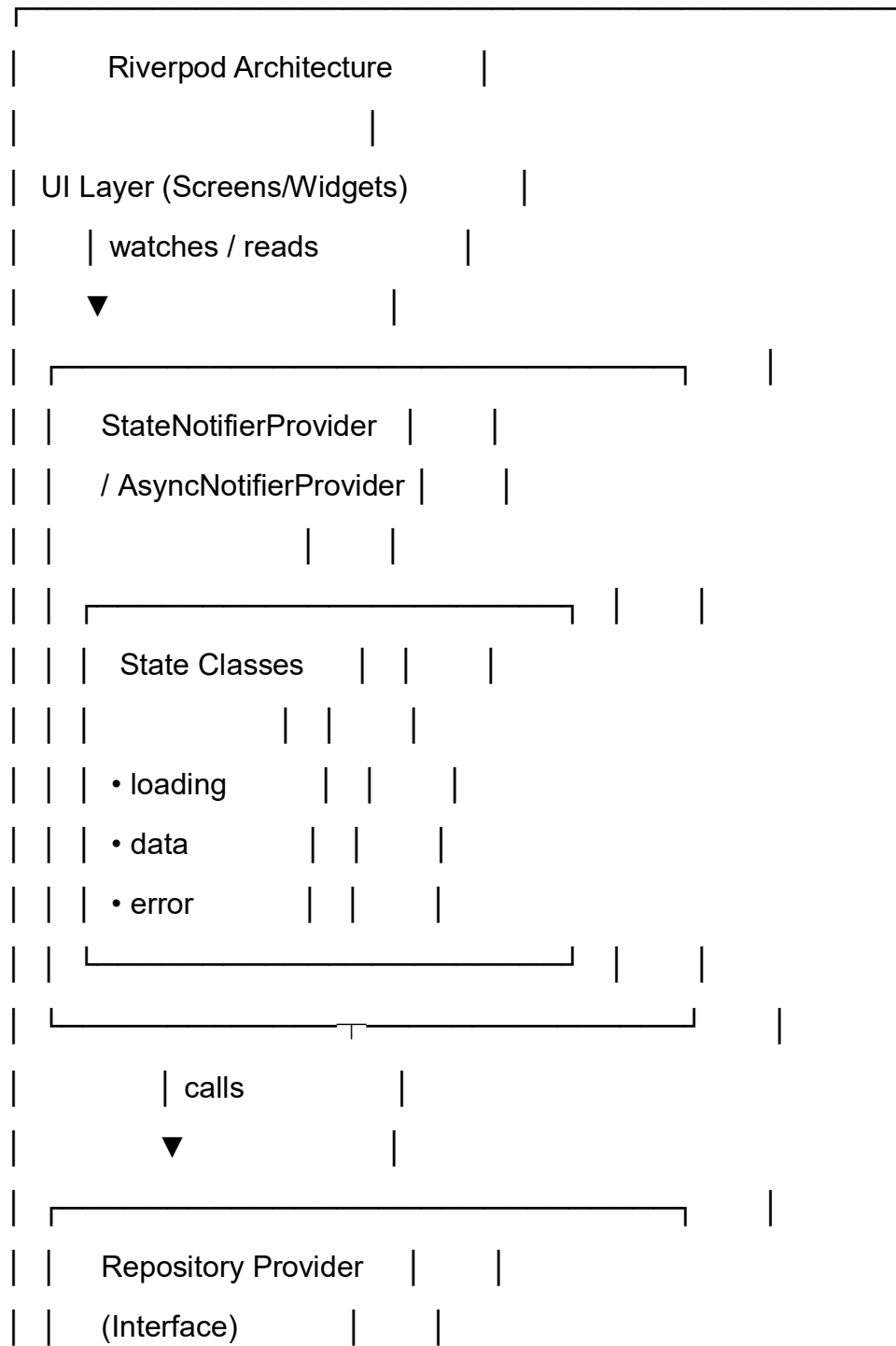


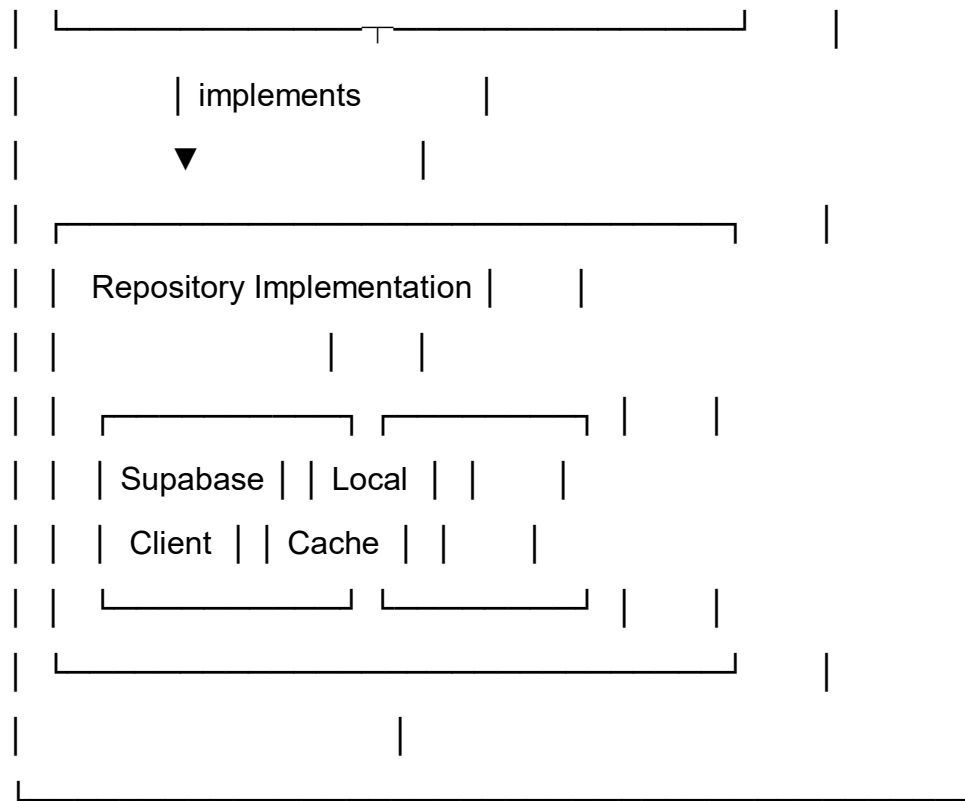
|               |             |   |   |  |
|---------------|-------------|---|---|--|
|               |             |   |   |  |
| YES           | NO          |   |   |  |
|               |             |   |   |  |
|               | ▼           |   |   |  |
|               |             |   |   |  |
| Show Error    |             |   |   |  |
|               |             |   |   |  |
|               |             |   |   |  |
| ▼             |             | ▼ | ▼ |  |
|               |             |   |   |  |
| Fetch Profile |             |   |   |  |
| (role check)  |             |   |   |  |
|               |             |   |   |  |
|               |             |   |   |  |
| ▼             |             |   |   |  |
| Role?         |             |   |   |  |
|               |             |   |   |  |
| user          | admin       |   |   |  |
|               |             |   |   |  |
| ▼             | ▼           |   |   |  |
|               |             |   |   |  |
| Home          | Admin Dash. |   |   |  |
|               |             |   |   |  |
|               |             |   |   |  |

...

## ## 3.6 State Management

...





...

---

## # 4. Functional Requirements

### ## 4.1 Authentication Module

#### ### FR-AUTH-001: User Registration

| Attribute | Details |

|-----|-----|

| **ID** | FR-AUTH-001 |

| **Title** | User Registration with Email and Password |

| **Priority** | High |

| **Description** | The system shall allow new users to create an account using their email address and a password |

**Input Fields:**

| Field | Type | Required | Validation |

|-----|-----|-----|-----|

| Full Name | String | Yes | 3-100 characters, letters and spaces only |

| Email | String | Yes | Valid email format, unique in system |

| Phone | String | No | Valid phone format with country code |

| Password | String | Yes | Min 8 chars, 1 uppercase, 1 number, 1 special |

| Confirm Password | String | Yes | Must match Password field |

| Terms Agreement | Boolean | Yes | Must be true |

**Processing:**

1. Validate all input fields client-side
2. Send registration request to Supabase Auth
3. Supabase creates auth.users entry
4. Database trigger auto-creates profiles entry
5. Return JWT session token
6. Navigate to Home Screen

**Error Handling:**

| Error | Message | Action |

|                                                                           |
|---------------------------------------------------------------------------|
| ----- ----- -----                                                         |
| Email exists   "This email is already registered"   Highlight email field |
| Weak password   "Password must contain..."   Show requirements            |
| Network error   "No internet connection"   Show retry option              |
| Server error   "Something went wrong"   Show retry option                 |

---

### ### FR-AUTH-002: User Login

|                                                                                                            |
|------------------------------------------------------------------------------------------------------------|
| Attribute   Details                                                                                        |
| ----- -----                                                                                                |
| <b>ID</b>   FR-AUTH-002                                                                                    |
| <b>Title</b>   User Login with Email/Password or Google                                                    |
| <b>Priority</b>   High                                                                                     |
| <b>Description</b>   The system shall allow registered users to login using email/password or Google OAuth |

#### **Method A — Email/Password:**

|                                           |
|-------------------------------------------|
| Field   Type   Required   Validation      |
| ----- ----- ----- -----                   |
| Email   String   Yes   Valid email format |
| Password   String   Yes   Non-empty       |

#### **Method B — Google Sign-In:**

1. User taps "Continue with Google"
2. Google OAuth flow opens
3. User selects Google account
4. Supabase receives OAuth token
5. If new user → auto-create profile
6. If existing user → login directly
7. Return JWT session token

---

### ### FR-AUTH-003: Password Reset

|              |                                  |
|--------------|----------------------------------|
| Attribute    | Details                          |
| -----        | -----                            |
| **ID**       | FR-AUTH-003                      |
| **Title**    | Forgot Password / Reset Password |
| **Priority** | Medium                           |

**\*\*Flow:\*\***

1. User enters registered email
2. System sends password reset email via Supabase
3. User clicks link in email
4. User enters new password
5. Password updated in system

---

### ### FR-AUTH-004: Logout

| Attribute | Details     |
|-----------|-------------|
| ID        | FR-AUTH-004 |
| Title     | User Logout |
| Priority  | High        |

#### **Flow:**

1. User taps Logout in Profile screen
2. Confirmation dialog appears
3. User confirms
4. Clear local session and cached data
5. Navigate to Login Screen

---

## ## 4.2 Home Module

### ### FR-HOME-001: Display Home Screen

| Attribute | Details                          |
|-----------|----------------------------------|
| ID        | FR-HOME-001                      |
| Title     | Display personalized Home Screen |

| **Priority** | High |

**Components:**

| Section | Data Source | Refresh |

|-----|-----|-----|

| Greeting Header | profiles table (user name) | On screen load |

| Search Bar | — (navigation trigger) | Static |

| Promotional Banner | meals where is\_featured = true | Every 30 min |

| Categories | categories table | On first load, cache |

| Popular Meals | meals ordered by total\_ratings DESC, limit 10 | Every 15 min |

| Recent Ratings | ratings where user\_id = current, limit 3 | Realtime |

---

## ## 4.3 Meals Module

### ### FR-MEAL-001: Browse Meals by Category

| Attribute | Details |

|-----|-----|

| **ID** | FR-MEAL-001 |

| **Title** | Browse and filter meals |

| **Priority** | High |

**Filtering Options:**

| Filter | Type | Default |

|-----|-----|-----|

| Category | Single select | All |

| Rating Range | Chip select (All / Top / Average / Poor) | All |

| New Arrivals | Toggle | Off |

**\*\*Sorting Options:\*\***

| Sort By | Direction |

|-----|-----|

| Most Popular | total\_ratings DESC |

| Highest Rated | avg\_overall DESC |

| Newest | created\_at DESC |

| Name A-Z | name ASC |

**\*\*Pagination:\*\***

- Page size: 20 meals per request
- Infinite scroll with loading indicator
- Pull-to-refresh support

---

### FR-MEAL-002: Search Meals

| Attribute | Details |

|-----|-----|

| **ID** | FR-MEAL-002 |

| **Title** | Search meals by name |

| **Priority** | Medium |

**Behavior:**

- Debounced search (300ms delay after typing stops)
- Search in meal name (English and Arabic)
- Minimum 2 characters to trigger search
- Show recent searches (stored locally)
- Show "No results" empty state when no matches

---

### FR-MEAL-003: View Meal Details

| Attribute | Details |

|-----|-----|

| **ID** | FR-MEAL-003 |

| **Title** | Display detailed meal information |

| **Priority** | High |

**Data Displayed:**

| Field | Source |

|-----|-----|

| Meal Image | meals.image\_url (Supabase Storage) |

|                  |                                                           |
|------------------|-----------------------------------------------------------|
| Meal Name        | meals.name                                                |
| Category         | categories.name via meals.category_id                     |
| Average Rating   | meals.avg_overall                                         |
| Total Ratings    | meals.total_ratings                                       |
| Total Comments   | COUNT(ratings where comment IS NOT NULL)                  |
| Description      | meals.description                                         |
| Rating Breakdown | meals.avg_taste, avg_presentation, avg_portion, avg_value |
| Recent Reviews   | ratings (latest 5, with profiles join)                    |
| Is Favorite      | favorites (check if user_id + meal_id exists)             |

---

## ## 4.4 Rating Module

### ### FR-RATE-001: Submit Meal Rating

|              |                                    |
|--------------|------------------------------------|
| Attribute    | Details                            |
| -----        | -----                              |
| **ID**       | FR-RATE-001                        |
| **Title**    | Rate a meal on multiple dimensions |
| **Priority** | High (Core Feature)                |

**\*\*Rating Dimensions:\*\***

|           |                            |       |          |
|-----------|----------------------------|-------|----------|
| Dimension | Input Type                 | Range | Required |
| -----     | -----                      | ----- | -----    |
| Taste     | Emoji selector (5 options) | 1-5   | Yes      |

| Presentation | Star rating (tap) | 1-5 | Yes |  
| Portion Size | Slider | 1.0-5.0 (0.5 step) | Yes |  
| Value for Money | Slider | 1.0-5.0 (0.5 step) | Yes |  
| Photo | Image picker | Max 5MB, JPG/PNG | No |  
| Comment | Text field | 0-300 characters | No |

#### **\*\*Overall Score Calculation:\*\***

...

$$\text{overall\_score} = (\text{taste} + \text{presentation} + \text{portion} + \text{value}) / 4.0$$

...

Calculated automatically in real-time as user adjusts ratings. Also enforced as a generated column in PostgreSQL.

#### **\*\*Processing Flow:\*\***

...

1. User opens Rating Screen for a specific meal
2. User rates each dimension
3. Overall score updates in real-time
4. User optionally adds photo and/or comment
5. User taps Submit
6. Client validates all required fields
7. If photo exists → upload to Supabase Storage → get URL
8. Insert rating record into ratings table
9. Database trigger updates meal averages

10. Create notification for user (rating\_confirmed)

11. Navigate to Success Screen

...

**\*\*Business Rules:\*\***

- One rating per user per meal (UNIQUE constraint on user\_id + meal\_id)
- If user already rated, open in edit mode with existing values
- Default slider values: 3.0 (middle)
-